

# Product Requirement Document (PRD): LocalLore

## Project Overview

The goal of this project is to develop **LocalLore**, an agentic web application that transforms standard geographical searches into immersive historical and cultural discoveries. Users enter a location using an autocomplete search bar, which immediately opens a dynamic, interactive map centered on that location.

To keep operational and infrastructural costs at exactly **\$0.00**, the system balances a dual-layer strategy: an instant geospatial lookup using **Supabase (PostgreSQL with PostGIS)**, and an asynchronous, throttled background queue using **Upstash QStash** powered by the **Google Gemini API Free Tier**.

If a search is a cache miss, a background job is securely enqueued, paced to respect free-tier rate limits, and executed asynchronously. Curated historical summaries are synthesized into structured JSON data, written back to the database, and populated dynamically onto the user's map canvas as custom pins.

**Level:** Medium

**Type of Project:** AI Agent Development, Asynchronous Architecture, Geospatial Web App

**Skills Required:** Next.js (React & TypeScript), Vercel AI SDK, Upstash QStash, Interactive Mapping APIs (Mapbox), Supabase (PostgreSQL & PostGIS)

## Technical Stack Architecture

- **Frontend & Backend Hosting:** Next.js (App Router) deployed on **Vercel (Hobby Tier)**.
- **Maps & Autocomplete:** Mapbox Search Box API (leveraging the 50,000 free monthly search sessions tier) paired with **Mapbox GL JS** for rendering the interactive map canvas.
- **Asynchronous Message Broker:** Upstash QStash (utilizing the 500 free daily messages tier) acting as an HTTP-push orchestrator to bypass serverless timeout limits.
- **Agent Framework & LLM:** Vercel AI SDK (`generateObject` schema enforcement) paired with **Google Gemini 2.5 Flash** (via a card-free, unbilled Google AI Studio project key).

- **Database & Caching Layer: Supabase (PostgreSQL + PostGIS)** to calculate bounding boxes and store stateful lore payloads.
- **Primary Data Source: Wikipedia API** ([geosearch](#) endpoint).

## Key Features & Milestones

### Milestone 1: Debounced Search & Instant Map Focus

- **Debounced Autocomplete Search:** A clean search bar with a 300ms debounce buffer to limit unnecessary API requests while typing. It suggests locations seamlessly via Mapbox.
- **Immediate Coordinate Handoff:** Selecting a suggestion immediately grabs the latitude and longitude, hides the search layout, and launches a full-screen interactive map centered on those coordinates.

### Milestone 2: Smart Caching Layer (Supabase PostGIS)

- **Geospatial Cache Lookup:** Before triggering any background actions, the Next.js API route queries Supabase using PostGIS to check if a user has searched within a specific radius (e.g., a 2km bounding box) of these coordinates recently.
- **Instant Cache Hit Delivery:** If a match is found in Supabase, the app bypasses the queue entirely, serving the saved lore data instantly to the map pins in under 50ms.

### Milestone 3: Decoupled HTTP-Push Queueing (Cache Miss)

- **Asynchronous Background Enqueueing:** If the location is a cache miss, the Next.js backend immediately inserts a placeholder "Pending" row into Supabase and dispatches an HTTP message to **Upstash QStash**. The endpoint immediately returns a **200 OK** status to the frontend client.
- **Concurrency Throttling:** QStash holds the job in a buffer, configured with a concurrency rate ceiling (e.g., maximum 5 parallel jobs) to safely protect the free-tier Gemini API key from bursting past its 15 Requests Per Minute (RPM) restriction.
- **The Serverless Execution Loop:** QStash wakes up a protected serverless Next.js endpoint ([/api/cron/process-lore](#)) by sending a secure HTTP POST signature. This route fetches nearby raw text summaries from the Wikipedia API in a single batch, calls Gemini 2.5 Flash to filter and synthesize the text into punchy 2-3 sentence hooks via Zod schemas, hydrates the Supabase cache table, and cleanly shuts down.

### Milestone 4: Interactive Map Exploration & Polling UI

- **Dynamic Ghost Pins:** While a background job is actively processing in the queue, the map displays a temporary, pulsing loading indicator over the target coordinates.

- **Real-time State Syncing:** The Next.js frontend utilizes a lightweight polling library (like SWR) or a native Supabase Realtime subscription to monitor the coordinate status row.
- **Lore Cards:** As soon as the background worker completes and saves the payload, the UI automatically replaces the loader with interactive map pins. Clicking a marker reveals an overlay card showcasing the snappy headline, the narrative hook, and a "Read More" button linking to the source Wikipedia page.

## Target Audience & UX Priorities

The app targets curious travelers, history buffs, and locals eager to explore the hidden background of their surroundings. Because processing moves entirely to the background, the UI prioritizes asynchronous user feedback—ensuring the application never freezes, crashes, or drops requests due to upstream cloud AI rate limitations.